

FTRIA-109 — Runtime Invariant Discovery Pipeline

Part II — Runtime Invariant Algebra

Sizhe Tan
with chatGPT (OpenAI)
2026-07-16

Repository: <https://github.com/sizhet/Function-Tunnel-and-Runtime-Invariant-Algebra-FTRIA>

Abstract

Runtime Invariant Discovery is often described in terms of individual algorithms such as Common Concept Core (CCC), Differential Trees, Transformer embeddings, or graph matching.

However, practical experience across ImageStarmap, GraphStarmap, SequenceStarmap, Structural Intelligence, and related systems suggests that these algorithms frequently share a common structural workflow.

This paper proposes the **Runtime Invariant Discovery Pipeline (RIDP)** as a unified interpretation of this workflow.

Rather than viewing Runtime Invariant Discovery as a single algorithm, RIDP describes a sequence of Runtime Operators that progressively transform multiple observations into validated Runtime Invariants.

This pipeline provides a common structural language for understanding both symbolic and neural approaches to Runtime Invariant Discovery.

1. Discovery as a Process Rather Than an Algorithm

Runtime Invariant Discovery should not be regarded as a single computational step.

Instead,

it is better understood as a sequence of structural transformations.

Conceptually,

Multiple Observations

↓

Alignment

↓

Consensus



Runtime Invariant



Validation



Certified Runtime Invariant

Each stage performs a distinct Runtime Function,
while together they construct reusable Runtime Knowledge.

2. Stage One — Runtime Alignment

The first stage identifies possible structural correspondences among multiple observations.

Its objective is not yet to determine the Runtime Invariant,
but to generate plausible structural alignments.

Typical implementations include

- Bucket Tree of Permutation (BTP),
- graph matching,
- sequence alignment,
- structural correspondence generation.

Within Runtime Invariant Algebra,
these algorithms perform a common Runtime Function:

Runtime Alignment.

Alignment generates candidate structural organizations upon which subsequent stages operate.

3. Stage Two — Structural Consensus

Multiple candidate alignments generally coexist.

The second stage evaluates these candidates collectively,
identifying structural regularities that consistently appear across different observations.

Typical implementations include

- voting,
- trimming,
- consensus formation,
- redundancy elimination.

This stage suppresses accidental variation while preserving stable structural relationships.

Consensus therefore represents the transition from candidate organization to structural stability.

4. Stage Three — Runtime Invariant Formation

Once structural consensus has emerged, the Runtime Invariant itself can be constructed.

Examples include

- Common Concept Core (CCC),
- structural prototypes,
- invariant representations,
- reusable semantic structures.

This stage transforms structural agreement into an explicit Runtime Invariant. Rather than representing individual observations, the Runtime Invariant captures the stable structure underlying all admissible realizations.

5. Stage Four — Runtime Invariant Validation

Discovery is incomplete until the proposed Runtime Invariant has been validated.

Validation evaluates whether the discovered Runtime Invariant adequately explains the original observations.

Typical validation mechanisms include

- Scoring Trees,
- Two-Way CCC,
- structural consistency checking,
- bidirectional verification,
- confidence estimation.

Validation distinguishes structural hypotheses from certified Runtime Knowledge.

6. The Complete Runtime Invariant Discovery Pipeline

The four stages form a unified Discovery Pipeline.

Observations

↓

Runtime Alignment

↓

Structural Consensus

↓

Runtime Invariant

↓

Runtime Validation

↓

Certified Runtime Invariant

Although different AI systems implement these stages differently, the underlying structural workflow remains remarkably consistent.

7. Relationship to Existing AI

Many existing AI systems perform portions of this pipeline.

Structural Intelligence

- BTP,
- CCC,
- Differential Trees,
- Scoring Trees.

Transformer-based AI

- contextual alignment,
- representation aggregation,
- embedding formation,
- prediction loss.

Vision AI

- feature correspondence,
- feature aggregation,
- object representation,
- recognition confidence.

Graph AI

- graph alignment,
- graph consensus,
- structural embedding,
- graph verification.

Different implementations,
yet a common Runtime Discovery Pipeline.

8. Engineering Implications

Viewing Runtime Invariant Discovery as a pipeline provides several advantages.

It separates

- candidate generation,
- structural organization,
- invariant construction,
- structural verification

into distinct Runtime Operators.

Consequently,

individual stages may be independently optimized,
extended,
or replaced,

while preserving the overall Discovery Architecture.

This modular perspective naturally supports plugin-based implementations,
cross-domain reuse,
and future Runtime Invariant Engineering.

9. Toward Runtime Structural Organization

Runtime Alignment constitutes the first stage of Discovery,
but not its final objective.

As Runtime Discovery becomes increasingly sophisticated,
Alignment gradually expands into broader forms of Structural Organization,
including

- hierarchy construction,
- Differential Trees,
- Universal Task Networks,
- Calling Graphs,
- Runtime Graphs.

This observation suggests that Runtime Alignment may eventually be
understood as a special case of a more general Runtime Structural
Organization Algebra,

a direction explored in subsequent FTRIA papers.

Key Takeaways

- Runtime Invariant Discovery is best understood as a structural pipeline rather than a single algorithm.
- The pipeline consists of four principal Runtime Operators: Alignment, Consensus, Runtime Invariant Formation, and Validation.
- Bucket Tree of Permutation (BTP) naturally serves as a Runtime Alignment implementation.
- Common Concept Core represents Runtime Invariant Formation.
- Scoring Trees and Two-Way CCC perform Runtime Validation.
- Different AI paradigms implement different versions of the same Discovery Pipeline.
- Runtime Alignment provides the foundation for the broader concept of Runtime Structural Organization.

Related FTRIA Documents

Discovery Algebra

- FTRIA-101 — Runtime Invariant Discovery Algebra
- FTRIA-102 — Common Concept Core (CCC) as Runtime Invariant Discovery
- FTRIA-106 — LLM Word Embeddings as Approximate Runtime Invariant Encoding
- FTRIA-107 — Transformer Attention as Dynamic Runtime Invariant Discovery

Future Discovery

- FTRIA-108 — Runtime Alignment Operators
- FTRIA-117 — Runtime Structural Organization Operators

Extension & Triggering

- FTRIA-110 — Runtime Invariant Extension Algebra
- FTRIA-120 — Runtime Invariant Trigger Algebra